

1) Descrição

Implementar um programa (em C ou C++) que simule a execução de instruções mostradas na tabela de decodificação fornecida na Seção 4. Considere que o processador possui 16 bits e os registradores R0-R7, PC, IR, SP e FLAGS.

2) Especificação

a) O simulador deverá receber como entrada um arquivo de texto contendo o código a ser executado, em notação hexadecimal. Este arquivo representa a memória de programa, onde cada linha corresponde a um endereço com alinhamento de 16 bits, no formato <endereço>:<conteúdo>.

Exemplo:

Código	Conteúdo do arquivo de texto
Inicio:	0000: 0x1803
MOV R0, #3	0002: 0x1905
MOV R1, #5	0004: 0x4220
ADD R2, R1, R0	0006: 0x0009
PSH R2	0008: 0x0702
POP R7	000A: 0x52E4
SUB R2, R7, R1	000C: 0x005F
CMP R2, R7	000E: 0x0FC1
JEQ Inicio	0010: 0x1040
MOV R0, R2	0012: 0x0FF8
fim:	
JMP fim	

b) O programa deve executar até encontrar uma instrução HALT OU uma instrução de formato indefinido OU atingir o final do arquivo.

c) Ao final da execução, o simulador deverá apresentar o conteúdo (em notação hexadecimal, indicada pelo prefixo '0x') dos seguintes componentes:

– *Registradores*
→ R0-R7, PC, e SP.

– *Memória de dados*

- No mesmo formato do arquivo de entrada (<endereço>:<conteúdo>)
- Assuma que toda a faixa de endereços está disponível e que a memória está inicialmente zerada. Devem ser exibidas apenas as posições que forem acessadas pelo código (desconsiderando a pilha).

– *Pilha*

- No mesmo formato do arquivo de entrada (<endereço>:<conteúdo>)
- Assuma que o ponteiro de pilha com o valor inicial de 0x8200 e que pilha possui tamanho de 16 bytes

– *Flags*

- Exiba os valores finais das flags C, Ov, Z, S
- Assuma que as todas iniciam zeradas

d) Sempre que ocorrer uma instrução NOP, o simulador deve exibir as mesmas informações descritas no *item c*.

3) Avaliação e Entrega

a) Cada trabalho será apresentado em sala e submetido a 3 casos de teste. Serão verificadas as informações mostradas ao final da execução (e sempre que ocorrer uma instrução NOP).

- Cada caso de teste vale de 0,0 a 3,0 pontos;
- A cada verificação será contabilizada uma pontuação igual a $3,0 / (n+1)$, onde n é o número de instruções NOP presentes no código.
- Será descontado 1,0 ponto para cada item que não exiba os valores corretos ou que não atenda **integralmente** as especificações, até no máximo 3,0 pontos por caso de teste.

b) Será adicionada a pontuação 0,0 a 1,0 conforme as características gerais do projeto, a critério do professor.

- Será atribuída nota total 0,0 para os trabalhos que apresentarem semelhanças com indícios de cópia.

c) A pontuação total da equipe será igual aos pontos obtidos multiplicados pelo total de membros da equipe. Essa pontuação deverá ser dividida entre os membros para compor a nota individual de cada um, a critério da equipe.

d) Entregar até dia **28/02/2025**

4) Tabela de decodificação

Instrução	Operação	Tipo	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
<u>NOP</u>	<u>nop</u>	<u>NOP</u>	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
<u>HALT</u>	<u>halt</u>	<u>HALT</u>	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
<u>MOV Rd, Rm</u>	<u>Rd = Rm</u>	<u>MOV</u>	0	0	0	1	0	Rd ₂	Rd ₁	Rd ₀	Rm ₂	Rm ₁	Rm ₀	-	-	-	-	-
<u>MOV Rd, #Im</u>	<u>Rd = #Im</u>	<u>MOV</u>	0	0	0	1	1	Rd ₂	Rd ₁	Rd ₀	Im ₇	Im ₆	Im ₅	Im ₄	Im ₃	Im ₂	Im ₁	Im ₀
<u>STR [Rm], Rn</u>	<u>[Rm] = Rn</u>	<u>STORE</u>	0	0	1	0	0	-	-	-	Rm ₂	Rm ₁	Rm ₀	Rn ₂	Rn ₁	Rn ₀	-	-
<u>STR [Rm], #Im</u>	<u>[Rm] = #Im</u>	<u>STORE</u>	0	0	1	0	1	Im ₇	Im ₆	Im ₅	Rm ₂	Rm ₁	Rm ₀	Im ₄	Im ₃	Im ₂	Im ₁	Im ₀
<u>LDR Rd, [Rm]</u>	<u>Rd = [Rm]</u>	<u>LOAD</u>	0	0	1	1	-	Rd ₂	Rd ₁	Rd ₀	Rm ₂	Rm ₁	Rm ₀	-	-	-	-	-
<u>ADD Rd, Rm, Rn</u>	<u>Rd = Rm + Rn</u>	<u>ULA</u>	0	1	0	0	-	Rd ₂	Rd ₁	Rd ₀	Rm ₂	Rm ₁	Rm ₀	Rn ₂	Rn ₁	Rn ₀	-	-
<u>SUB Rd, Rm, Rn</u>	<u>Rd = Rm - Rn</u>	<u>ULA</u>	0	1	0	1	-	Rd ₂	Rd ₁	Rd ₀	Rm ₂	Rm ₁	Rm ₀	Rn ₂	Rn ₁	Rn ₀	-	-
<u>MUL Rd, Rm, Rn</u>	<u>Rd = Rm * Rn</u>	<u>ULA</u>	0	1	1	0	-	Rd ₂	Rd ₁	Rd ₀	Rm ₂	Rm ₁	Rm ₀	Rn ₂	Rn ₁	Rn ₀	-	-
<u>AND Rd, Rm, Rn</u>	<u>Rd = Rm and Rn</u>	<u>ULA</u>	0	1	1	1	-	Rd ₂	Rd ₁	Rd ₀	Rm ₂	Rm ₁	Rm ₀	Rn ₂	Rn ₁	Rn ₀	-	-
<u>ORR Rd, Rm, Rn</u>	<u>Rd = Rm or Rn</u>	<u>ULA</u>	1	0	0	0	-	Rd ₂	Rd ₁	Rd ₀	Rm ₂	Rm ₁	Rm ₀	Rn ₂	Rn ₁	Rn ₀	-	-
<u>NOT Rd, Rm</u>	<u>Rd = ~Rm</u>	<u>ULA</u>	1	0	0	1	-	Rd ₂	Rd ₁	Rd ₀	Rm ₂	Rm ₁	Rm ₀	-	-	-	-	-
<u>XOR Rd, Rm, Rn</u>	<u>Rd = Rm xor Rn</u>	<u>ULA</u>	1	0	1	0	-	Rd ₂	Rd ₁	Rd ₀	Rm ₂	Rm ₁	Rm ₀	Rn ₂	Rn ₁	Rn ₀	-	-
<u>PSH Rn</u>	<u>[SP] = Rn; SP --</u>	<u>PILHA</u>	0	0	0	0	0	-	-	-	-	-	-	Rn ₂	Rn ₁	Rn ₀	0	1
<u>POP Rd</u>	<u>SP++; Rd = [SP]</u>	<u>PILHA</u>	0	0	0	0	0	Rd ₂	Rd ₁	Rd ₀	-	-	-	-	-	-	1	0
<u>CMP Rm, Rn</u>	<u>Z = (Rm = Rn)? 1 : 0 ; C = (Rm < Rn)? 1 : 0</u>	<u>ULA</u>	0	0	0	0	0	-	-	-	Rm ₂	Rm ₁	Rm ₀	Rn ₂	Rn ₁	Rn ₀	1	1
<u>JMP #Im</u>	<u>PC = PC + #Im</u>	<u>DESVIO</u>	0	0	0	0	1	Im ₈	Im ₇	Im ₆	Im ₅	Im ₄	Im ₃	Im ₂	Im ₁	Im ₀	0	0
<u>JEQ #Im</u>	<u>PC = PC + #Im, se Z = 1 e C = 0</u>	<u>DESVIO</u>	0	0	0	0	1	Im ₈	Im ₇	Im ₆	Im ₅	Im ₄	Im ₃	Im ₂	Im ₁	Im ₀	0	1
<u>JLT #Im</u>	<u>PC = PC + #Im, se Z = 0 e C = 1</u>	<u>DESVIO</u>	0	0	0	0	1	Im ₈	Im ₇	Im ₆	Im ₅	Im ₄	Im ₃	Im ₂	Im ₁	Im ₀	1	0
<u>JGT #Im</u>	<u>PC = PC + #Im, se Z = 0 e C = 0</u>	<u>DESVIO</u>	0	0	0	0	1	Im ₈	Im ₇	Im ₆	Im ₅	Im ₄	Im ₃	Im ₂	Im ₁	Im ₀	1	1
<u>SHR Rd, Rm, #Im</u>	<u>Rd = Rm >> #Im</u>	<u>ULA</u>	1	0	1	1	-	Rd ₂	Rd ₁	Rd ₀	Rm ₂	Rm ₁	Rm ₀	Im ₄	Im ₃	Im ₂	Im ₁	Im ₀
<u>SHL Rd, Rm, #Im</u>	<u>Rd = Rm << #Im</u>	<u>ULA</u>	1	1	0	0	-	Rd ₂	Rd ₁	Rd ₀	Rm ₂	Rm ₁	Rm ₀	Im ₄	Im ₃	Im ₂	Im ₁	Im ₀
<u>ROR Rd, Rm</u>	<u>Rd = Rm >> 1; Rd(MSB) = Rm(LSB)</u>	<u>ULA</u>	1	1	0	1	-	Rd ₂	Rd ₁	Rd ₀	Rm ₂	Rm ₁	Rm ₀	-	-	-	-	-
<u>ROL Rd, Rm</u>	<u>Rd = Rm << 1; Rd(LSB) = Rm(MSB)</u>	<u>ULA</u>	1	1	1	0	-	Rd ₂	Rd ₁	Rd ₀	Rm ₂	Rm ₁	Rm ₀	-	-	-	-	-